

✓ **Nom et Prénom :** BOUKHRIS Mohamed Lyazid

TP Deep Learning - CNN

Cette troisième séance porte sur la découverte des réseaux de convolutions :

- Retour sur MNIST
- Première architecture convolutionnelle
- Visualisation des filtres
- Utilisation de VGG16, un réseau pré-entraîné
- Visualisation des filtres par maximisation des activations
- Augmentation de données
- Classification du jeu de données Cifar10

Import des premiers packages nécessaires :

```
import numpy as np
import tensorflow as tf
from tensorflow import keras as keras
from tensorflow.keras.models import Sequential
from tensorflow.keras.datasets import mnist
from tensorflow.keras.layers import Dense, Flatten, Dropout, Conv2D, MaxPooling2D, BatchNormalization
print(keras.__version__)
```

3.10.0

Chargement des données

Ici on charge le jeu de données MNIST (les chiffres manuscrits), puis quelques instructions de mise en forme

```
(x_train, y_train), (x_test, y_test) = mnist.load_data()
img_rows, img_cols = 28, 28
nb_classes = 10
nb_samples = len(x_train)
x_train = x_train.reshape(x_train.shape[0], img_rows, img_cols, 1) # (batch, rows, cols, channel)
x_test = x_test.reshape(x_test.shape[0], img_rows, img_cols, 1)
input_shape = (img_rows, img_cols, 1) # shape d'un élément, utile pour keras
x_train = x_train.astype('float32')
x_test = x_test.astype('float32')
x_train /= 255
x_test /= 255
```

Le jeu de données MNIST contient 60000 exemples répartis en 10 classes. Nous allons ici en sélectionner aléatoirement un certain nombre pour accélérer les étapes suivantes.

```
nb_subsamples = 10000
l_idx = np.random.permutation(list(range(nb_samples)))[:nb_subsamples]

x_train, y_train = x_train[l_idx], y_train[l_idx]
```

Il est d'abord nécessaire de formater les vecteurs d'étiquettes en *one-hot vectors* de tailles num_classes. Ces vecteurs contiennent des 0 et un seul 1 par ligne à l'indice correspondant à la classe. Tensorflow/Keras a une fonction pour ça :

```
from tensorflow.keras.utils import to_categorical
y_train2 = to_categorical(y_train, nb_classes)
y_test2 = to_categorical(y_test, nb_classes)
print(y_train[0], y_train2[0]) # affiche une classe et sa représentation one_hot
```

```
8 [0. 0. 0. 0. 0. 0. 0. 0. 1. 0.]
```

Avec Numpy, ça revient à :

```
y_train2 = np.zeros((nb_subsamples, nb_classes))
y_train2[np.arange(nb_subsamples), y_train] = 1
print(y_train[0], y_train2[0]) # affiche une classe et sa représentation one_hot
```

Keras/Tensorflow

Création d'un réseau de neurones

Avec Keras, Tensorflow propose deux API pour la définition de réseaux de neurones. La plus simple *Sequential* est illustrée dans ce notebook. Elle permet de créer de simples architectures construits comme une séquence de couches.

Pour les architectures plus complexes, la *Graph API* permet de définir un graphe de couches avec plusieurs entrées et sorties. La cellule suivante montre un exemple minimal :

```
from tensorflow.keras.models import Model
from tensorflow.keras.layers import Input, Dense, Concatenate

# branche 1
input1 = Input(shape=(392,)) # entrée 1 : vecteur de dimension 392
x1 = Dense(256, activation='relu')(input1)
x1 = Dense(512, activation='relu')(x1)

# branche 2
input2 = Input(shape=(392,)) # entrée 1 : vecteur de dimension 392
x2 = Dense(256, activation='relu')(input2)
x2 = Dense(512, activation='relu')(x2)

# merge and output
x = Concatenate()([x1, x2])
x = Dense(512, activation='relu')(x)
x = Dense(10, activation='softmax')(x)

# instantiate and compile model
model = Model([input1, input2], [x]) # Model(liste d'entrées, liste de sorties)
model.compile(loss="categorical_crossentropy", optimizer="adam", metrics=["accuracy"])
model.summary()
```

Model: "functional_74"

Layer (type)	Output Shape	Param #	Connected to
input_layer_30 (InputLayer)	(None, 392)	0	-
input_layer_31 (InputLayer)	(None, 392)	0	-
dense_41 (Dense)	(None, 256)	100,608	input_layer_30[0]...
dense_43 (Dense)	(None, 256)	100,608	input_layer_31[0]...
dense_42 (Dense)	(None, 512)	131,584	dense_41[0][0]
dense_44 (Dense)	(None, 512)	131,584	dense_43[0][0]
concatenate_1 (Concatenate)	(None, 1024)	0	dense_42[0][0], dense_44[0][0]
dense_45 (Dense)	(None, 512)	524,800	concatenate_1[0]...
dense_46 (Dense)	(None, 10)	5,130	dense_45[0][0]

Total params: 994,314 (3.79 MB)
Trainable params: 994,314 (3.79 MB)
Non-trainable params: 0 (0.00 B)

Entraînement "haut niveau" de Keras : fit, predict, evaluate

Keras offre plusieurs niveaux d'implémentations d'une boucle d'apprentissage, le plus haut niveau reprend le modèle de sklearn avec la fonction fit. Comme on le verra à la fin de notebook, fit fonctionne aussi avec des générateurs python, par exemple pour l'augmentation de données.

Réseau Dense

Prenons un réseau MLP simple et notez son nombre de paramètres et sa performance obtenue, ils nous serviront de référence. La couche Flatten mise en entrée du réseau sert à vectoriser les images MNIST puisque les couches Dense opèrent uniquement sur des vecteurs.

```
model1 = Sequential() # API Sequential de Keras pour des réseaux simples
model1.add(Flatten(input_shape=input_shape)) # input_shape est seulement nécessaire sur la première couche
model1.add(Dense(512, activation='relu'))
```

```

model1.add(Dropout(0.2))
model1.add(Dense(1024, activation='relu'))
model1.add(Dropout(0.2))
model1.add(Dense(nb_classes, activation='softmax'))
model1.compile(loss='categorical_crossentropy', optimizer='adam', metrics=['accuracy'])

```

```
print(model1.summary())
```

```

loss = model1.fit(x_train, y_train2, batch_size=128, epochs=20, validation_split=0.1, callbacks=[], verbose=1)
score = model1.evaluate(x_test, y_test2, verbose=0)

```

```
print("score=", score) # score est un tuple (loss, metric)
```

```

/usr/local/lib/python3.12/dist-packages/keras/src/layers/reshaping/flatten.py:37: UserWarning: Do not pass an `input_shape`/`inp
super().__init__(**kwargs)

```

```
Model: "sequential_27"
```

Layer (type)	Output Shape	Param #
flatten_25 (Flatten)	(None, 784)	0
dense_47 (Dense)	(None, 512)	401,920
dropout_40 (Dropout)	(None, 512)	0
dense_48 (Dense)	(None, 1024)	525,312
dropout_41 (Dropout)	(None, 1024)	0
dense_49 (Dense)	(None, 10)	10,250

```
Total params: 937,482 (3.58 MB)
```

```
Trainable params: 937,482 (3.58 MB)
```

```
Non-trainable params: 0 (0.00 B)
```

```
None
```

```
Epoch 1/20
```

```
71/71 ————— 6s 39ms/step - accuracy: 0.6863 - loss: 1.0035 - val_accuracy: 0.9150 - val_loss: 0.2958
```

```
Epoch 2/20
```

```
71/71 ————— 0s 4ms/step - accuracy: 0.9271 - loss: 0.2478 - val_accuracy: 0.9380 - val_loss: 0.2234
```

```
Epoch 3/20
```

```
71/71 ————— 0s 4ms/step - accuracy: 0.9557 - loss: 0.1443 - val_accuracy: 0.9290 - val_loss: 0.2194
```

```
Epoch 4/20
```

```
71/71 ————— 0s 4ms/step - accuracy: 0.9685 - loss: 0.1052 - val_accuracy: 0.9480 - val_loss: 0.1823
```

```
Epoch 5/20
```

```
71/71 ————— 0s 4ms/step - accuracy: 0.9762 - loss: 0.0686 - val_accuracy: 0.9460 - val_loss: 0.1806
```

```
Epoch 6/20
```

```
71/71 ————— 0s 4ms/step - accuracy: 0.9854 - loss: 0.0511 - val_accuracy: 0.9470 - val_loss: 0.1822
```

```
Epoch 7/20
```

```
71/71 ————— 0s 4ms/step - accuracy: 0.9862 - loss: 0.0399 - val_accuracy: 0.9530 - val_loss: 0.1747
```

```
Epoch 8/20
```

```
71/71 ————— 0s 4ms/step - accuracy: 0.9915 - loss: 0.0334 - val_accuracy: 0.9420 - val_loss: 0.2211
```

```
Epoch 9/20
```

```
71/71 ————— 0s 4ms/step - accuracy: 0.9914 - loss: 0.0305 - val_accuracy: 0.9520 - val_loss: 0.1665
```

```
Epoch 10/20
```

```
71/71 ————— 0s 5ms/step - accuracy: 0.9950 - loss: 0.0150 - val_accuracy: 0.9510 - val_loss: 0.2147
```

```
Epoch 11/20
```

```
71/71 ————— 1s 6ms/step - accuracy: 0.9948 - loss: 0.0178 - val_accuracy: 0.9430 - val_loss: 0.2172
```

```
Epoch 12/20
```

```
71/71 ————— 0s 6ms/step - accuracy: 0.9957 - loss: 0.0176 - val_accuracy: 0.9480 - val_loss: 0.1886
```

```
Epoch 13/20
```

```
71/71 ————— 0s 6ms/step - accuracy: 0.9967 - loss: 0.0126 - val_accuracy: 0.9520 - val_loss: 0.2062
```

```
Epoch 14/20
```

```
71/71 ————— 0s 6ms/step - accuracy: 0.9969 - loss: 0.0104 - val_accuracy: 0.9500 - val_loss: 0.2133
```

```
Epoch 15/20
```

```
71/71 ————— 0s 4ms/step - accuracy: 0.9961 - loss: 0.0121 - val_accuracy: 0.9500 - val_loss: 0.2375
```

```
Epoch 16/20
```

```
71/71 ————— 0s 5ms/step - accuracy: 0.9931 - loss: 0.0230 - val_accuracy: 0.9460 - val_loss: 0.2468
```

```
Epoch 17/20
```

```
71/71 ————— 0s 4ms/step - accuracy: 0.9864 - loss: 0.0441 - val_accuracy: 0.9450 - val_loss: 0.2148
```

```
Epoch 18/20
```

```
71/71 ————— 0s 4ms/step - accuracy: 0.9949 - loss: 0.0175 - val_accuracy: 0.9530 - val_loss: 0.1998
```

```
Epoch 19/20
```

```
71/71 ————— 0s 4ms/step - accuracy: 0.9968 - loss: 0.0127 - val_accuracy: 0.9460 - val_loss: 0.2652
```

```
Epoch 20/20
```

```
71/71 ————— 0s 4ms/step - accuracy: 0.9964 - loss: 0.0115 - val_accuracy: 0.9500 - val_loss: 0.2232
```

```
score= [0.17385341227054596, 0.9624000191688538]
```

A Faire : Évaluez l'architecture à deux entrées définie plus haut sur les données MNIST en séparant en deux vecteurs de tailles 392 les vecteurs MNIST initiaux.

```

# Aplatissement des images MNIST
x_train3 = x_train.reshape((nb_subsamples, -1))
x_test3 = x_test.reshape((nb_subsamples, -1))

```

```

# Séparation en deux moitiés de 392
x_train_left = x_train3[:, :392]

```

```

x_train_right = x_train3[:, 392:]
x_test_left = x_test3[:, :392]
x_test_right = x_test3[:, 392:]

# Entraînement du modèle à deux entrées
loss = model.fit(
    [x_train_left, x_train_right],
    y_train2,
    batch_size=128,
    epochs=10,
    validation_split=0.1,
    verbose=1
)

# Évaluation sur le jeu de test
score = model.evaluate([x_test_left, x_test_right], y_test2, verbose=0)

print("score=", score) # score est un tuple (loss, metric)

```

```

Epoch 1/10
71/71 ————— 5s 33ms/step - accuracy: 0.7108 - loss: 0.9594 - val_accuracy: 0.9190 - val_loss: 0.2956
Epoch 2/10
71/71 ————— 0s 5ms/step - accuracy: 0.9441 - loss: 0.1929 - val_accuracy: 0.9480 - val_loss: 0.2072
Epoch 3/10
71/71 ————— 0s 5ms/step - accuracy: 0.9665 - loss: 0.1102 - val_accuracy: 0.9460 - val_loss: 0.1832
Epoch 4/10
71/71 ————— 0s 5ms/step - accuracy: 0.9807 - loss: 0.0657 - val_accuracy: 0.9470 - val_loss: 0.1872
Epoch 5/10
71/71 ————— 0s 5ms/step - accuracy: 0.9849 - loss: 0.0430 - val_accuracy: 0.9550 - val_loss: 0.1627
Epoch 6/10
71/71 ————— 0s 6ms/step - accuracy: 0.9909 - loss: 0.0292 - val_accuracy: 0.9560 - val_loss: 0.1670
Epoch 7/10
71/71 ————— 0s 6ms/step - accuracy: 0.9951 - loss: 0.0211 - val_accuracy: 0.9580 - val_loss: 0.1693
Epoch 8/10
71/71 ————— 0s 6ms/step - accuracy: 0.9952 - loss: 0.0145 - val_accuracy: 0.9500 - val_loss: 0.2152
Epoch 9/10
71/71 ————— 0s 7ms/step - accuracy: 0.9933 - loss: 0.0217 - val_accuracy: 0.9490 - val_loss: 0.1997
Epoch 10/10
71/71 ————— 1s 7ms/step - accuracy: 0.9943 - loss: 0.0170 - val_accuracy: 0.9530 - val_loss: 0.1956
score= [0.15906290709972382, 0.9595000147819519]

```

J'ai évalué l'architecture à deux vecteurs 392 sur MNIST. On constate que l'entraînement est proche de 100% (97,13%), la validation plafonne est 95% et 96% et le test est à 97%, ce qui montre que le modèle généralise bien, même si il y'a découpage en deux vecteurs et une couche denses. On remarque aussi un léger surapprentissage.

Première architecture convolutionnelle

Créons un nouveau réseau dont les couches Dense (sauf la dernière pour la classification) sont remplacées par des Conv2D, suivis de MaxPooling2D. Notons que la couche "Flatten" n'est pas utilisée en début de réseau, puisque les données du réseau sont supposées 2D. En revanche, la couche Dense en bout de réseau suppose des données d'entrée en 1D.

```

def simple_cnn():
    model2 = Sequential()
    model2.add(Conv2D(64, kernel_size=(5,5), strides=(1,1), padding='same', activation='relu', input_shape=input_shape))
    model2.add(MaxPooling2D((2,2)))
    model2.add(Conv2D(128, kernel_size=(3,3), strides=(1,1), padding='same', activation='relu'))
    model2.add(MaxPooling2D((2,2)))
    model2.add(Conv2D(128, kernel_size=(3,3), strides=(1,1), padding='same', activation='relu', kernel_regularizer='l2'))
    model2.add(MaxPooling2D((2,2)))
    model2.add(Flatten()) # Vectorise les features maps pour entrer dans une couche dense
    model2.add(Dropout(0.2))
    model2.add(Dense(nb_classes, activation='softmax'))
    model2.compile(loss='categorical_crossentropy', optimizer='adam', metrics=['accuracy'])
    print(model2.summary())
    return model2

```

A faire : Expliquez les nombres de paramètres de chaque couche.

- Les couches **Conv2D** comportent des paramètres qui correspondent aux poids et aux biais de chaque filtre, plus la couche contient de filtres ou plus les images d'entrée ont de canaux plus le nombre de paramètres est élevé.
- Les couches **MaxPooling**, **Flatten** et **Dropout** n'ont aucun paramètre car elle ne font que transformer ou réduire les données sans apprentissage.
- La couche **Dense** finale contient des paramètres qui correspondent aux connexions entre tous les neurones en entrée et les 10 neurones de sortie ainsi qu'un biais par neurone de sortie

Entrainement du réseau

Appelons maintenant les fonctions fit et evaluate pour entrainer et tester le réseau. Visualisez la courbe d'apprentissage pour vérifier que tout s'apprend bien.

```
model2 = simple_cnn()
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/convolutional/base_conv.py:113: UserWarning: Do not pass an `input_shape`  
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)  
Model: "sequential_28"
```

Layer (type)	Output Shape	Param #
conv2d_88 (Conv2D)	(None, 28, 28, 64)	1,664
max_pooling2d_71 (MaxPooling2D)	(None, 14, 14, 64)	0
conv2d_89 (Conv2D)	(None, 14, 14, 128)	73,856
max_pooling2d_72 (MaxPooling2D)	(None, 7, 7, 128)	0
conv2d_90 (Conv2D)	(None, 7, 7, 128)	147,584
max_pooling2d_73 (MaxPooling2D)	(None, 3, 3, 128)	0
flatten_26 (Flatten)	(None, 1152)	0
dropout_42 (Dropout)	(None, 1152)	0
dense_50 (Dense)	(None, 10)	11,530

Total params: 234,634 (916.54 KB)

Trainable params: 234,634 (916.54 KB)

Non-trainable params: 0 (0.00 B)

None

```
loss = model2.fit(x_train, y_train2, batch_size=128, epochs=20, validation_split=0.1, callbacks=[], verbose=1)  
score = model2.evaluate(x_test, y_test2, verbose=0)  
print("score=", score)
```

```
Epoch 1/20  
71/71 ━━━━━━━━━━━ 7s 57ms/step - accuracy: 0.5593 - loss: 2.1794 - val_accuracy: 0.9350 - val_loss: 0.5767  
Epoch 2/20  
71/71 ━━━━━━━━━━━ 1s 11ms/step - accuracy: 0.9426 - loss: 0.5096 - val_accuracy: 0.9450 - val_loss: 0.3808  
Epoch 3/20  
71/71 ━━━━━━━━━━━ 1s 11ms/step - accuracy: 0.9523 - loss: 0.3239 - val_accuracy: 0.9670 - val_loss: 0.2486  
Epoch 4/20  
71/71 ━━━━━━━━━━━ 1s 11ms/step - accuracy: 0.9642 - loss: 0.2338 - val_accuracy: 0.9640 - val_loss: 0.1974  
Epoch 5/20  
71/71 ━━━━━━━━━━━ 1s 11ms/step - accuracy: 0.9706 - loss: 0.1838 - val_accuracy: 0.9710 - val_loss: 0.1731  
Epoch 6/20  
71/71 ━━━━━━━━━━━ 1s 10ms/step - accuracy: 0.9741 - loss: 0.1716 - val_accuracy: 0.9740 - val_loss: 0.1513  
Epoch 7/20  
71/71 ━━━━━━━━━━━ 1s 10ms/step - accuracy: 0.9795 - loss: 0.1348 - val_accuracy: 0.9720 - val_loss: 0.1603  
Epoch 8/20  
71/71 ━━━━━━━━━━━ 1s 10ms/step - accuracy: 0.9789 - loss: 0.1345 - val_accuracy: 0.9710 - val_loss: 0.1404  
Epoch 9/20  
71/71 ━━━━━━━━━━━ 1s 10ms/step - accuracy: 0.9774 - loss: 0.1253 - val_accuracy: 0.9790 - val_loss: 0.1293  
Epoch 10/20  
71/71 ━━━━━━━━━━━ 1s 9ms/step - accuracy: 0.9824 - loss: 0.1117 - val_accuracy: 0.9740 - val_loss: 0.1413  
Epoch 11/20  
71/71 ━━━━━━━━━━━ 1s 10ms/step - accuracy: 0.9835 - loss: 0.1087 - val_accuracy: 0.9730 - val_loss: 0.1248  
Epoch 12/20  
71/71 ━━━━━━━━━━━ 1s 10ms/step - accuracy: 0.9815 - loss: 0.1070 - val_accuracy: 0.9690 - val_loss: 0.1341  
Epoch 13/20  
71/71 ━━━━━━━━━━━ 1s 10ms/step - accuracy: 0.9810 - loss: 0.1176 - val_accuracy: 0.9660 - val_loss: 0.1563  
Epoch 14/20  
71/71 ━━━━━━━━━━━ 1s 10ms/step - accuracy: 0.9828 - loss: 0.1098 - val_accuracy: 0.9760 - val_loss: 0.1256  
Epoch 15/20  
71/71 ━━━━━━━━━━━ 1s 10ms/step - accuracy: 0.9811 - loss: 0.1055 - val_accuracy: 0.9760 - val_loss: 0.1274  
Epoch 16/20  
71/71 ━━━━━━━━━━━ 1s 10ms/step - accuracy: 0.9875 - loss: 0.0846 - val_accuracy: 0.9750 - val_loss: 0.1221  
Epoch 17/20  
71/71 ━━━━━━━━━━━ 1s 10ms/step - accuracy: 0.9869 - loss: 0.0913 - val_accuracy: 0.9800 - val_loss: 0.1233  
Epoch 18/20  
71/71 ━━━━━━━━━━━ 1s 10ms/step - accuracy: 0.9851 - loss: 0.0926 - val_accuracy: 0.9720 - val_loss: 0.1261  
Epoch 19/20  
71/71 ━━━━━━━━━━━ 1s 12ms/step - accuracy: 0.9842 - loss: 0.0967 - val_accuracy: 0.9790 - val_loss: 0.1106  
Epoch 20/20  
71/71 ━━━━━━━━━━━ 1s 11ms/step - accuracy: 0.9863 - loss: 0.0934 - val_accuracy: 0.9790 - val_loss: 0.1196  
score= [0.08739149570465088, 0.9871000051498413]
```

A faire : Qu'observez-vous ? Comment l'expliquer ?

- J'observe que le réseau convolutionnel apprend très bien avec une précision d'entraînement proche de 99% et une précision de validation vers les 98%, la courbe d'apprentissage a une bonne convergence, on constate aussi un début de surapprentissage lorsque qu'il y'a eu une légère augmentation de loss de validation à la fin. Le modèle généralise bien avec une accuracy de 98% et cela s'explique par le fait que les couches convolutionnelles exploitent la structure spatiale des images et ils détectent de manière efficace les formes et les motifs caractéristiques des chiffres manuscrits.

Implémentation de la boucle d'entraînement

La seule fonction fit ne permet pas tout, il faut souvent aller plus en détail dans la boucle d'entraînement. Avec Keras, on peut utiliser la fonction `train_on_batch` :

```
# reinitialiation du modèle
model2 = simple_cnn()

# boucle d'entraînement
nb_epochs = 20
batch_size = 128
nb_batches = int(nb_subsamples/batch_size)

# Prepare les batches avec tf
train_dataset = tf.data.Dataset.from_tensor_slices((x_train, y_train2))
train_dataset = train_dataset.shuffle(buffer_size=1024).batch(batch_size)

for epoch in range(nb_epochs):
    for step, (x, y) in enumerate(train_dataset):
        loss = model2.train_on_batch(x, y)

    # affiche la loss du dernier batch à chaque fin d'epoch
    print("epoch", epoch, "loss=", loss)

score = model2.evaluate(x_test, y_test2, verbose=0)
print("score=", score)
```

Model: "sequential_29"

Layer (type)	Output Shape	Param #
conv2d_91 (Conv2D)	(None, 28, 28, 64)	1,664
max_pooling2d_74 (MaxPooling2D)	(None, 14, 14, 64)	0
conv2d_92 (Conv2D)	(None, 14, 14, 128)	73,856
max_pooling2d_75 (MaxPooling2D)	(None, 7, 7, 128)	0
conv2d_93 (Conv2D)	(None, 7, 7, 128)	147,584
max_pooling2d_76 (MaxPooling2D)	(None, 3, 3, 128)	0
flatten_27 (Flatten)	(None, 1152)	0
dropout_43 (Dropout)	(None, 1152)	0
dense_51 (Dense)	(None, 10)	11,530

Total params: 234,634 (916.54 KB)

Trainable params: 234,634 (916.54 KB)

Non-trainable params: 0 (0.00 B)

None

```
epoch 0 loss= [array(1.348149, dtype=float32), array(0.76140004, dtype=float32)]
epoch 1 loss= [array(0.8924763, dtype=float32), array(0.85435003, dtype=float32)]
epoch 2 loss= [array(0.6888172, dtype=float32), array(0.8894, dtype=float32)]
epoch 3 loss= [array(0.5685281, dtype=float32), array(0.90927505, dtype=float32)]
epoch 4 loss= [array(0.489325, dtype=float32), array(0.92161995, dtype=float32)]
epoch 5 loss= [array(0.43552044, dtype=float32), array(0.9300167, dtype=float32)]
epoch 6 loss= [array(0.3945359, dtype=float32), array(0.9360857, dtype=float32)]
epoch 7 loss= [array(0.3615822, dtype=float32), array(0.94121253, dtype=float32)]
epoch 8 loss= [array(0.33543697, dtype=float32), array(0.94515556, dtype=float32)]
epoch 9 loss= [array(0.31443164, dtype=float32), array(0.94852996, dtype=float32)]
epoch 10 loss= [array(0.29565126, dtype=float32), array(0.9515727, dtype=float32)]
epoch 11 loss= [array(0.28140098, dtype=float32), array(0.95367503, dtype=float32)]
epoch 12 loss= [array(0.26731032, dtype=float32), array(0.9561385, dtype=float32)]
epoch 13 loss= [array(0.25557202, dtype=float32), array(0.95805, dtype=float32)]
epoch 14 loss= [array(0.24488336, dtype=float32), array(0.95988, dtype=float32)]
epoch 15 loss= [array(0.23536354, dtype=float32), array(0.9614813, dtype=float32)]
epoch 16 loss= [array(0.22692534, dtype=float32), array(0.96289414, dtype=float32)]
epoch 17 loss= [array(0.21935968, dtype=float32), array(0.9641945, dtype=float32)]
epoch 18 loss= [array(0.21228538, dtype=float32), array(0.9653895, dtype=float32)]
epoch 19 loss= [array(0.20660183, dtype=float32), array(0.96630996, dtype=float32)]
score= [0.10423775762319565, 0.9818000197410583]
```

Boucle d'entrainement plus détaillée avec gradients

Pour aller encore plus loin, on peut explicitement avoir accès aux gradients et mettre à jours les paramètres du réseau. Accéder aux gradients permet par exemple de régulariser l'entrainement par "gradient clipping".

```
# reinitialiation du modèle
model2 = simple_cnn()

# fonction losse pour l'entrainement du modèle
loss_fun = tf.keras.losses.CategoricalCrossentropy()

for epoch in range(nb_epochs):
    mmin = 10000.
    mmax = -10000.
    for step, (x, y) in enumerate(train_dataset):

        # L'objet GradientTape permet de conserver les
        # with tf.GradientTape() as tape:

            # forward le batch -> obtient les logits
            logits = model2(x) # ajouter un true si je voulais le dropout

            # calcule la loss pour ce batch
            loss_value = loss_fun(y, logits)

        # calcule le gradient de la losse par rapport aux paramètres du modèle et mise à jour du modèle
        grads = tape.gradient(loss_value, model2.trainable_weights)

        clipped_grads = [
            tf.clip_by_value(g, -0.1, 0.1) if g is not None else None
            for g in grads
        ]
        # on filtre les None (certains poids peuvent ne pas avoir de gradient)
        pairs = [(g, w) for g, w in zip(clipped_grads, model2.trainable_weights) if g is not None]

        #model2.optimizer.apply_gradients(zip(grads, model2.trainable_weights))

        model2.optimizer.apply_gradients(pairs)

    # affiche la loss du dernier batch à chaque fin d'epoch
    print("epoch", epoch, "loss", loss_value.numpy()) #, "stats", mmin.numpy(), mmax.numpy())

score = model2.evaluate(x_test, y_test2, verbose=0)
print("score=", score)
```

Model: "sequential_30"

Layer (type)	Output Shape	Param #
conv2d_94 (Conv2D)	(None, 28, 28, 64)	1,664
max_pooling2d_77 (MaxPooling2D)	(None, 14, 14, 64)	0
conv2d_95 (Conv2D)	(None, 14, 14, 128)	73,856
max_pooling2d_78 (MaxPooling2D)	(None, 7, 7, 128)	0
conv2d_96 (Conv2D)	(None, 7, 7, 128)	147,584
max_pooling2d_79 (MaxPooling2D)	(None, 3, 3, 128)	0
flatten_28 (Flatten)	(None, 1152)	0
dropout_44 (Dropout)	(None, 1152)	0
dense_52 (Dense)	(None, 10)	11,530

Total params: 234,634 (916.54 KB)

Trainable params: 234,634 (916.54 KB)

Non-trainable params: 0 (0.00 B)

None

epoch 0 loss 0.084295064

epoch 1 loss 0.13324663

epoch 2 loss 0.029438416

epoch 3 loss 0.004329952

epoch 4 loss 0.108181484

epoch 5 loss 0.016846305

epoch 6 loss 0.0024610697

epoch 7 loss 0.0008752849

epoch 8 loss 0.0003315623

epoch 9 loss 0.00016964442

epoch 10 loss 0.00061477633

epoch 11 loss 0.005291926

epoch 12 loss 0.014031429

epoch 13 loss 0.000121479316

epoch 14 loss 0.00019443932

epoch 15 loss 0.00093789655

epoch 16 loss 2.1904432e-06

epoch 17 loss 2.72272e-05

epoch 18 loss 7.087737e-05

epoch 19 loss 3.756379e-05

score= [2.4619481563568115, 0.9890999794006348]

A faire : Ajoutez une régularisation par troncage du gradient (entre -0.1 et 0.1) et mesurez son impact. (grads est une liste de gradients de chaque couche)

- J'ai utilisé une régularisation par troncage du gradient en limitant les valeurs entre -0.1 et 0.1, cette méthode permet d'éviter que les gradients deviennent trop grands et rendent l'apprentissage instable. Après l'entraînement le modèle atteint 98,9% de précision sur le test presque le même résultat qu'avant le troncage. Cela montre que le troncage des gradients rend l'apprentissage plus stable sans avoir un effet sur la performance du réseau

Visualisation des cartes

Une carte "réponse" (ou feature map) d'une couche de convolution correspond à l'état des activations en sortie d'une telle couche. Leur contenu dépend donc d'une image passée en entrée du réseau. Une couche de convolution définie pour apprendre 64 filtres de convolution, génère ainsi 64 cartes.

Une manière simple pour obtenir ces cartes, via l'API Keras, consiste à faire passer une image (au choix mais de la bonne taille) à travers un réseau tronqué à la couche dont on veut visualiser les sorties.

```
from tensorflow.keras.models import Model

# Définition d'un modèle via la "graph" API de Keras
# model_tmp est le modèle tronqué après sa première couche
model_tmp = Model(model2.layers[0].input, model2.layers[0].output)

# Cartes réponses de x_test[10] (un '0')
feature_maps = model_tmp.predict(np.expand_dims(x_test[100], 0))

# normalisation des valeurs entre 0 et 1
minimum, maximum = np.min(feature_maps), np.max(feature_maps)
feature_maps = (feature_maps - minimum) / (maximum - minimum)
```

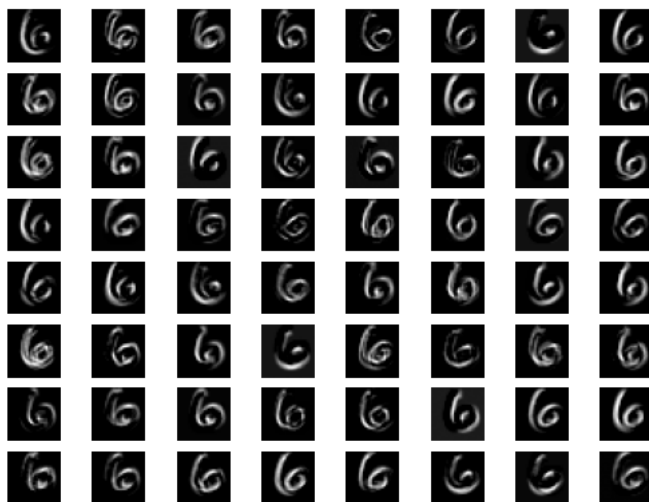


```
print(feature_maps.shape)
```

```
1/1 ————— 0s 228ms/step  
(1, 28, 28, 64)
```

On définit d'abord une fonction pour l'affichage d'un tableau d'images (via matplotlib) puis on convertit les features maps en "images".

```
import matplotlib.pyplot as plt  
from PIL import Image  
from IPython.display import display  
  
def show_images(images, cols = 1):  
    n_images = len(images)  
    fig = plt.figure()  
    for n, image in enumerate(images):  
        # a = fig.add_subplot(cols, np.ceil(n_images/float(cols)), n + 1)  
        a = fig.add_subplot(int(np.ceil(n_images/float(cols))), cols, n + 1)  
  
        plt.axis('off')  
        if image.ndim == 2:  
            plt.gray()  
            plt.imshow(image)  
  
# conversion features maps en images niveau de gris  
images = []  
for i in range(64):  
    images.append(np.array(255*feature_maps[:, :, :, i]).reshape(28,28).astype('uint8'))  
  
show_images(images, 8)
```



Les images obtenues semblent montrer que les filtres appris dans la première couche du réseau permettent la détection d'orientations ou des contours.

Visualisation des filtres

La visualisation des filtres de la couche d'entrée est relativement immédiate puisque que les filtres ont le même nombre de channels que les données d'entrée. Autrement dit, les filtres de la première couche sont définis sur le même domaine que les données.

```
# Récupère tous les paramètres appris par le réseau  
weights = model2.get_weights()  
  
for w in weights: print(w.shape)  
  
(5, 5, 1, 64)  
(64,)  
(3, 3, 64, 128)  
(128,)  
(3, 3, 128, 128)  
(128,)  
(1152, 10)  
(10,)
```

A faire : A quoi correspondent chaque ligne issues de l'affichage de la cellule précédente ?

Chaque ligne correspond aux poids et biais appris par le réseau.

- La première ligne (5,5,1,64) représente les 64 filtres 5x5 de la première couche de convolution appliqués sur les images en noir et blanc (1 canal).
- La deuxième ligne (64,) correspond aux biais de cette même couche.
- (3,3,64,128) et (128,) sont les poids et biais de la deuxième couche de convolution, qui prend en entrée les 64 cartes de la couche précédente et apprend 128 filtres.
- Les deux lignes suivantes (3,3,128,128) et (128,) concernent la troisième couche de convolution.
- Enfin, (1152,10) et (10,) sont les poids et biais de la couche Dense finale, qui relie les 1152 neurones de la couche flatten aux 10 classes de sortie du dataset MNIST.

Affichons maintenant les filtres de la première couche de convolution :

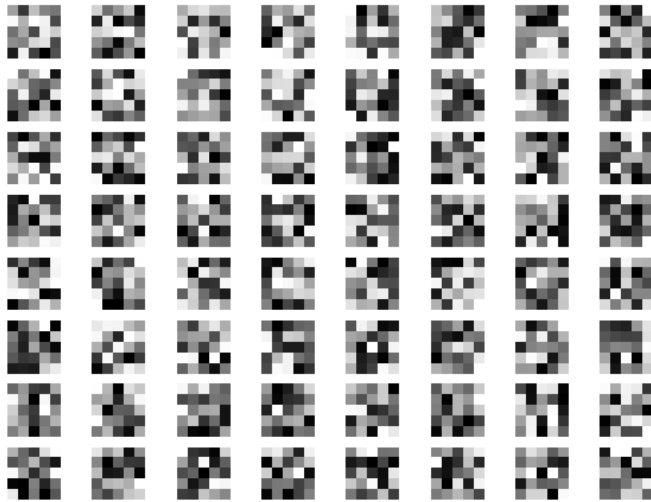
```
# normalize values between 0 and 1
minimum, maximum = np.min(weights[0]), np.max(weights[0])
weights0 = (weights[0] - minimum) / (maximum - minimum)

print(minimum, maximum)
# entre 0 et 255 pour l'affichage
weights0 *= 255.

images = []
for i in range(64):
    images.append(np.array(255*weights0[:, :, :, i]).reshape(5,5).astype('uint8'))

show_images(images, 8)
```

-0.3059731 0.17455606



CNN pour des images naturelles

Les filtres de convolutions appris sur les données MNIST ne sont pas très parlantes, intéressons nous plutôt à un réseau (beaucoup plus profond) adapté à la classification d'images naturelles. Nous allons considérer pour cela le réseau VGG16, pré-entraîné sur le dataset ImageNet (cf cours).

```
!wget https://pageperso.lis-lab.fr/stephane.ayache/cat.jpg
!wget https://pageperso.lis-lab.fr/stephane.ayache/imagenet_class_index.json

from tensorflow.keras.applications import VGG16
from tensorflow.keras.preprocessing import image
from tensorflow.keras.applications.vgg16 import preprocess_input
import json

# Récupération du modèle complet VGG16
model3 = VGG16(include_top=True, weights='imagenet')
print(model3.summary())

# chargement d'un dictionnaire qui met en correspondant l'index d'une classe ImageNet et son nom
with open('imagenet_class_index.json') as f:
    CLASS_INDEX = json.load(f)

# chargement et preprocess de l'image (dont normalisation et redimensionnement)
img_path = 'cat.jpg'
img = image.load_img(img_path, target_size=(224, 224))
x = image.img_to_array(img)
x = np.expand_dims(x, axis=0)
x = preprocess_input(x)
```

```
# prédiction et affichage de la classe de probabilité maximale
softmax_output = model3.predict(x)
best_class = np.argmax(softmax_output)
im_class = CLASS_INDEX[str(best_class)][1]
print("prediction: ", im_class)
```

```
--2025-10-21 20:51:45-- https://pageperso.lis-lab.fr/stephane.ayache/cat.jpg
Resolving pageperso.lis-lab.fr (pageperso.lis-lab.fr)... 139.124.22.27
Connecting to pageperso.lis-lab.fr (pageperso.lis-lab.fr)|139.124.22.27|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 86130 (84K) [image/jpeg]
Saving to: 'cat.jpg.1'
```

```
cat.jpg.1          100%[=====>] 84.11K  215KB/s   in 0.4s
```

```
2025-10-21 20:51:47 (215 KB/s) - 'cat.jpg.1' saved [86130/86130]
```

```
--2025-10-21 20:51:47-- https://pageperso.lis-lab.fr/stephane.ayache/imagenet_class_index.json
Resolving pageperso.lis-lab.fr (pageperso.lis-lab.fr)... 139.124.22.27
Connecting to pageperso.lis-lab.fr (pageperso.lis-lab.fr)|139.124.22.27|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 35363 (35K) [application/json]
Saving to: 'imagenet_class_index.json.1'
```

```
imagenet_class_inde 100%[=====>] 34.53K  180KB/s   in 0.2s
```

```
2025-10-21 20:51:48 (180 KB/s) - 'imagenet_class_index.json.1' saved [35363/35363]
```

Model: "vgg16"

Layer (type)	Output Shape	Param #
input_layer_36 (InputLayer)	(None, 224, 224, 3)	0
block1_conv1 (Conv2D)	(None, 224, 224, 64)	1,792
block1_conv2 (Conv2D)	(None, 224, 224, 64)	36,928
block1_pool (MaxPooling2D)	(None, 112, 112, 64)	0
block2_conv1 (Conv2D)	(None, 112, 112, 128)	73,856
block2_conv2 (Conv2D)	(None, 112, 112, 128)	147,584
block2_pool (MaxPooling2D)	(None, 56, 56, 128)	0
block3_conv1 (Conv2D)	(None, 56, 56, 256)	295,168
block3_conv2 (Conv2D)	(None, 56, 56, 256)	590,080
block3_conv3 (Conv2D)	(None, 56, 56, 256)	590,080
block3_pool (MaxPooling2D)	(None, 28, 28, 256)	0
block4_conv1 (Conv2D)	(None, 28, 28, 512)	1,180,160
block4_conv2 (Conv2D)	(None, 28, 28, 512)	2,359,808
block4_conv3 (Conv2D)	(None, 28, 28, 512)	2,359,808
block4_pool (MaxPooling2D)	(None, 14, 14, 512)	0
block5_conv1 (Conv2D)	(None, 14, 14, 512)	2,359,808
block5_conv2 (Conv2D)	(None, 14, 14, 512)	2,359,808
block5_conv3 (Conv2D)	(None, 14, 14, 512)	2,359,808
block5_pool (MaxPooling2D)	(None, 7, 7, 512)	0
flatten (Flatten)	(None, 25088)	0
fc1 (Dense)	(None, 4096)	102,764,544
fc2 (Dense)	(None, 4096)	16,781,312
predictions (Dense)	(None, 1000)	4,097,000

Total params: 138,357,544 (527.79 MB)
Trainable params: 138,357,544 (527.79 MB)
Non-trainable params: 0 (0.00 B)

```
None
1/1 ----- 1s 939ms/step
prediction: tiger_cat
```

Notre chaton a été reconnu comme un chat ? Alors tout va bien, et regardons les filtres de la première couche. texte en gras

```

weights = model3.get_weights()
for w in weights: print(w.shape)

# normalize values between 0 and 1
minimum, maximum = np.min(weights[0]), np.max(weights[0])
weights0 = (weights[0] - minimum) / (maximum - minimum)

print(minimum, maximum)
# entre 0 et 255 pour l'affichage
weights0 *= 255.

images = []
for i in range(64):
    images.append(np.array(255*weights0[:, :, :, i]).reshape(3,3,3).astype('uint8'))

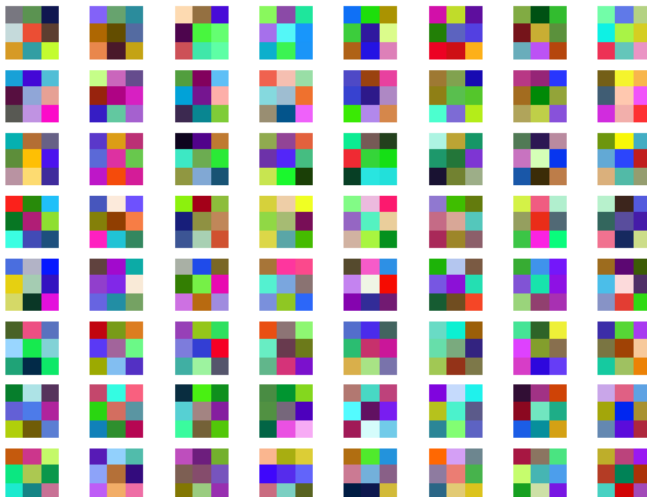
show_images(images, 8)

```

```

(3, 3, 3, 64)
(64,)
(3, 3, 64, 64)
(64,)
(3, 3, 64, 128)
(128,)
(3, 3, 128, 128)
(128,)
(3, 3, 128, 256)
(256,)
(3, 3, 256, 256)
(256,)
(3, 3, 256, 256)
(256,)
(3, 3, 256, 512)
(512,)
(3, 3, 512, 512)
(512,)
(3, 3, 512, 512)
(512,)
(3, 3, 512, 512)
(512,)
(3, 3, 512, 512)
(512,)
(3, 3, 512, 512)
(512,)
(25088, 4096)
(4096,)
(4096, 4096)
(4096,)
(4096, 1000)
(1000,)
-0.67140007 0.6085159

```



Notez qu'on affiche ici les filtres en RGB alors qu'on pourrait les afficher en séparant les channels. Dans tous les cas, cette visualisation reste peu informative. Pour mieux comprendre le rôle d'une couche de convolution, d'autres méthodes sont plus efficaces. Ci dessous, nous allons générer une image qui maximise la valeur d'une carte d'activation qui nous permettra d'interpréter ce que reconnaît un filtre dans le domaine des pixels.

Entraînement d'un réseau plus profond

Un réseau très profond tel que VGG contient énormément de paramètres, son entraînement peu s'avérer compliqué. Tous les paramètres doivent entrer en mémoire (RAM ou GPU), ainsi que toutes les données d'un minibatch. Dans le cas d'images relativement volumineuses

(ie: Imagenet =255x255x3), la taille du minibatch est souvent ainsi très couteux en mémoire et se voit ainsi réduit à quelques images, rendant l'entrainement encore plus long...

De plus, de grandes bases de données ne sont pas toujours disponibles. Dans ce cas, pour entrainer un tel réseau, plusieurs options sont possibles :

- partir d'un réseau déjà (pré-)entrainé, et/ou utiliser ses poids pour initialiser un autre réseau qui sera raffiné sur une base (réduite) d'images (= finetuning)
- régulariser les poids du réseau (peut aider mais ne suffit pas)
- augmenter artificiellement le nombre de données (améliore toujours les performances d'un réseau)

Dans la suite, nous illustrons ce dernier point sur les données MNIST.

```
from tensorflow import keras as keras
from tensorflow.keras.models import Sequential
from tensorflow.keras.datasets import mnist
from tensorflow.keras.layers import Dense, Flatten, Dropout, Conv2D, MaxPooling2D, BatchNormalization

(x_train, y_train), (x_test, y_test) = mnist.load_data()
img_rows, img_cols = 28, 28
num_classes = 10
nb_samples = len(x_train)
x_train = x_train.reshape(x_train.shape[0], img_rows, img_cols, 1)
x_test = x_test.reshape(x_test.shape[0], img_rows, img_cols, 1)
input_shape = (img_rows, img_cols, 1)
x_train = x_train.astype('float32')
x_test = x_test.astype('float32')
x_train /= 255
x_test /= 255

# Sélection aléatoire de 5000 exemples
l_idx = list(range(nb_samples))
np.random.shuffle(l_idx)
l_idx = l_idx[:500]
x_train, y_train = x_train[l_idx], y_train[l_idx]

# conversion des étiquettes au format one-hot vector
y_train = keras.utils.to_categorical(y_train, num_classes)
y_test = keras.utils.to_categorical(y_test, num_classes)

# Architecture du modèle
model2 = Sequential()
model2.add(Conv2D(64, kernel_size=(5,5), strides=(1,1), padding='same', activation='relu', input_shape=input_shape))
model2.add(MaxPooling2D((2,2)))
model2.add(Conv2D(128, kernel_size=(3,3), strides=(1,1), padding='same', activation='relu'))
model2.add(MaxPooling2D((2,2)))
model2.add(Conv2D(128, kernel_size=(3,3), strides=(1,1), padding='same', activation='relu'))
model2.add(MaxPooling2D((2,2)))
model2.add(Flatten())
model2.add(Dropout(0.2))
model2.add(Dense(num_classes, activation='softmax'))
model2.compile(loss='categorical_crossentropy', optimizer='adam', metrics=['accuracy'])

#print(model2.summary())

model2.fit(x_train, y_train, batch_size=128, epochs=20, verbose=1)

score = model2.evaluate(x_test, y_test, verbose=0)
print("score=", score)
```

```
Epoch 1/20
4/4 ————— 6s 789ms/step - accuracy: 0.1576 - loss: 2.2795
Epoch 2/20
4/4 ————— 0s 17ms/step - accuracy: 0.5405 - loss: 2.0767
Epoch 3/20
4/4 ————— 0s 18ms/step - accuracy: 0.5782 - loss: 1.6684
Epoch 4/20
4/4 ————— 0s 18ms/step - accuracy: 0.7099 - loss: 1.1480
Epoch 5/20
4/4 ————— 0s 17ms/step - accuracy: 0.7982 - loss: 0.6714
Epoch 6/20
4/4 ————— 0s 18ms/step - accuracy: 0.8080 - loss: 0.5788
Epoch 7/20
4/4 ————— 0s 18ms/step - accuracy: 0.8737 - loss: 0.3666
Epoch 8/20
4/4 ————— 0s 18ms/step - accuracy: 0.8990 - loss: 0.2953
Epoch 9/20
4/4 ————— 0s 18ms/step - accuracy: 0.8976 - loss: 0.2721
Epoch 10/20
4/4 ————— 0s 19ms/step - accuracy: 0.9527 - loss: 0.1873
Epoch 11/20
4/4 ————— 0s 17ms/step - accuracy: 0.9348 - loss: 0.1789
```

```

Epoch 12/20
4/4 ————— 0s 18ms/step - accuracy: 0.9592 - loss: 0.1538
Epoch 13/20
4/4 ————— 0s 21ms/step - accuracy: 0.9558 - loss: 0.1108
Epoch 14/20
4/4 ————— 0s 18ms/step - accuracy: 0.9674 - loss: 0.0951
Epoch 15/20
4/4 ————— 0s 17ms/step - accuracy: 0.9648 - loss: 0.0972
Epoch 16/20
4/4 ————— 0s 17ms/step - accuracy: 0.9772 - loss: 0.0663
Epoch 17/20
4/4 ————— 0s 17ms/step - accuracy: 0.9756 - loss: 0.0703
Epoch 18/20
4/4 ————— 0s 18ms/step - accuracy: 0.9926 - loss: 0.0412
Epoch 19/20
4/4 ————— 0s 18ms/step - accuracy: 0.9929 - loss: 0.0435
Epoch 20/20
4/4 ————— 0s 17ms/step - accuracy: 0.9850 - loss: 0.0448
score= [0.22733911871910095, 0.9315999746322632]

```

La performance obtenue peut être améliorée en augmentant les données. La cellule suivante génère 20000 données à partir des 5000 utilisées jusque là.

```

from tensorflow.keras.preprocessing.image import ImageDataGenerator

# déclaration d'un générateur, qui transformera "à la volée" des données MNIST
datagen = ImageDataGenerator(
    rotation_range=10,
    zoom_range = 0.05,
    width_shift_range=0.05,
    height_shift_range=0.05,
    horizontal_flip=False, # flip horizontal et vertical n'ont pas de sens pour des digits !
    vertical_flip=False
)
datagen.fit(x_train)

# On instancie le générateur
flow = datagen.flow(x_train, y_train, batch_size=128, shuffle=True)

# Pour affichage : une itération du générateur
xx = next(flow)
print(xx[0].shape, xx[1].shape)

images = []
for i in range(64):
    images.append(np.array(255*xx[0][i,:,:,:]).reshape(28,28).astype('uint8'))
show_images(images, 8)

# réinitialisation du modèle
model2 = Sequential()
model2.add(Conv2D(64, kernel_size=(5,5), strides=(1,1), padding='same', activation='relu', input_shape=input_shape))
model2.add(MaxPooling2D((2,2)))
model2.add(Conv2D(128, kernel_size=(3,3), strides=(1,1), padding='same', activation='relu'))
model2.add(MaxPooling2D((2,2)))
model2.add(Conv2D(128, kernel_size=(3,3), strides=(1,1), padding='same', activation='relu'))
model2.add(MaxPooling2D((2,2)))
model2.add(Flatten())
model2.add(Dropout(0.2))
model2.add(Dense(num_classes, activation='softmax'))
model2.compile(loss='categorical_crossentropy', optimizer='adam', metrics=['accuracy'])

# entraînement avec les données augmentées (à la volée)
#model2.fit_generator(datagen.flow(x_train, y_train, batch_size=128), steps_per_epoch=int(20000/128), epochs=20)
model2.fit(flow, steps_per_epoch=int(20000/128), epochs=20)

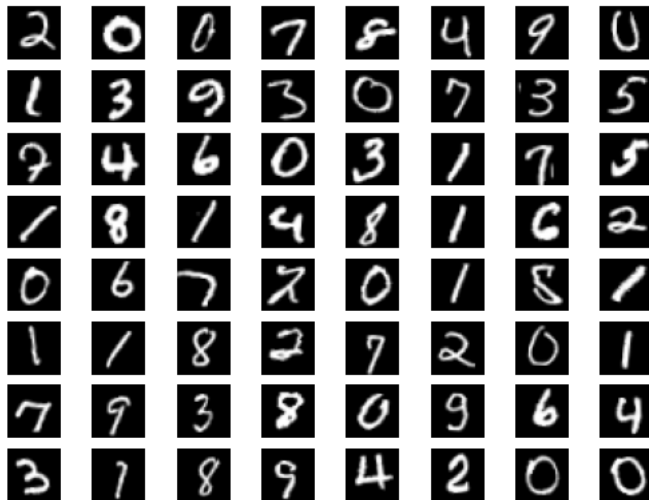
# évaluation
score = model2.evaluate(x_test, y_test, verbose=0)
print("score=", score)

```

```

(128, 28, 28, 1) (128, 10)
Epoch 1/20
/usr/local/lib/python3.12/dist-packages/keras/src/trainers/data_adapters/py_dataset_adapter.py:121: UserWarning: Your `PyDataset
self._warn_if_super_not_called()
156/156 ━━━━━━━━━━━ 7s 20ms/step - accuracy: 0.2430 - loss: 2.2546
Epoch 2/20
3/156 ━━━━━━━━━━━ 6s 44ms/step - accuracy: 0.3546 - loss: 2.1087/usr/local/lib/python3.12/dist-packages/keras/src/tra
self._interrupted_warning()
156/156 ━━━━━━━━━━━ 0s 1ms/step - accuracy: 0.3854 - loss: 2.0476
Epoch 3/20
156/156 ━━━━━━━━━━━ 0s 961us/step - accuracy: 0.5488 - loss: 1.6303
Epoch 4/20
156/156 ━━━━━━━━━━━ 0s 920us/step - accuracy: 0.6417 - loss: 1.1841
Epoch 5/20
156/156 ━━━━━━━━━━━ 0s 904us/step - accuracy: 0.7197 - loss: 0.8477
Epoch 6/20
156/156 ━━━━━━━━━━━ 0s 961us/step - accuracy: 0.6977 - loss: 0.8854
Epoch 7/20
156/156 ━━━━━━━━━━━ 0s 1ms/step - accuracy: 0.7821 - loss: 0.6543
Epoch 8/20
156/156 ━━━━━━━━━━━ 0s 956us/step - accuracy: 0.7781 - loss: 0.6214
Epoch 9/20
156/156 ━━━━━━━━━━━ 0s 904us/step - accuracy: 0.8296 - loss: 0.4973
Epoch 10/20
156/156 ━━━━━━━━━━━ 0s 990us/step - accuracy: 0.8658 - loss: 0.4276
Epoch 11/20
156/156 ━━━━━━━━━━━ 0s 953us/step - accuracy: 0.8678 - loss: 0.4054
Epoch 12/20
156/156 ━━━━━━━━━━━ 0s 912us/step - accuracy: 0.8842 - loss: 0.3443
Epoch 13/20
156/156 ━━━━━━━━━━━ 0s 923us/step - accuracy: 0.9037 - loss: 0.2907
Epoch 14/20
156/156 ━━━━━━━━━━━ 0s 972us/step - accuracy: 0.9119 - loss: 0.2864
Epoch 15/20
156/156 ━━━━━━━━━━━ 0s 952us/step - accuracy: 0.9042 - loss: 0.2553
Epoch 16/20
156/156 ━━━━━━━━━━━ 0s 1ms/step - accuracy: 0.9319 - loss: 0.2046
Epoch 17/20
156/156 ━━━━━━━━━━━ 0s 935us/step - accuracy: 0.9379 - loss: 0.1786
Epoch 18/20
156/156 ━━━━━━━━━━━ 0s 949us/step - accuracy: 0.9420 - loss: 0.1782
Epoch 19/20
156/156 ━━━━━━━━━━━ 0s 933us/step - accuracy: 0.9499 - loss: 0.1655
Epoch 20/20
156/156 ━━━━━━━━━━━ 0s 1ms/step - accuracy: 0.9264 - loss: 0.2034
score= [0.1777874082326889, 0.9462000131607056]

```



A faire : testez d'autres formes d'augmentation et d'autres quantités pour constater l'impact sur la précision du réseau.

```

# Comparaison d'augmentations et de quantités (MNIST)
# Objectif : je mesure l'impact de plusieurs formes d'augmentation ET de plusieurs quantités par époque
# sur la précision test, en gardant le même modèle et les mêmes hyperparamètres pour une comparaison juste.

import numpy as np
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
from tensorflow.keras.preprocessing.image import ImageDataGenerator

# Chargement et prétraitement
# J'ai normalisé en [0,1] et ajouté une dimension canal (grayscale) pour le CNN.
(x_train, y_train), (x_test, y_test) = keras.datasets.mnist.load_data()
x_train = x_train.astype("float32")/255.0

```

```

x_test = x_test.astype("float32")/255.0
x_train = np.expand_dims(x_train, -1) # (N,28,28,1)
x_test = np.expand_dims(x_test, -1)

# J'ai encodé les labels en one-hot pour la loss categorical_crossentropy.
num_classes = 10
y_train = keras.utils.to_categorical(y_train, num_classes)
y_test = keras.utils.to_categorical(y_test, num_classes)

# Échantillonnage contrôlé
# J'ai choisi de partir de 500 images réelles pour rendre l'augmentation "visible" dans les résultats.
nb_samples = 500
idx = np.arange(len(x_train))
np.random.shuffle(idx)
x_train_small = x_train[idx[:nb_samples]]
y_train_small = y_train[idx[:nb_samples]]

def build_mnist_cnn():
    # J'ai gardé une architecture CNN simple et identique pour toutes les expériences
    # (Conv-ReLU + MaxPool répété, puis Dense softmax). Ainsi, seule l'augmentation change.
    model = keras.Sequential([
        layers.Conv2D(64,(5,5),padding='same',activation='relu',input_shape=(28,28,1)),
        layers.MaxPooling2D(),
        layers.Conv2D(128,(3,3),padding='same',activation='relu'),
        layers.MaxPooling2D(),
        layers.Conv2D(128,(3,3),padding='same',activation='relu'),
        layers.MaxPooling2D(),
        layers.Flatten(),
        layers.Dropout(0.2), # -> J'ai ajouté un léger dropout pour limiter l'overfit.
        layers.Dense(num_classes, activation='softmax')
    ])
    model.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])
    return model

# Politiques d'augmentation
# J'ai défini trois niveaux : light / medium / heavy.
# Je n'utilise pas de flip pour des chiffres, car inverser horizontalement peut changer le sens du digit.
aug = {
    "none": ImageDataGenerator(), # -> Baseline sans augmentation, pour référence.
    "light": ImageDataGenerator(
        rotation_range=10,
        width_shift_range=0.05, height_shift_range=0.05,
    ),
    "medium": ImageDataGenerator(
        rotation_range=15,
        width_shift_range=0.1, height_shift_range=0.1,
        zoom_range=0.1, shear_range=10
    ),
    "heavy": ImageDataGenerator(
        rotation_range=25,
        width_shift_range=0.15, height_shift_range=0.15,
        zoom_range=0.2, shear_range=20
        # flip pas pertinent pour des digits
    ),
}

# Quantités virtuelles générées par époque
# J'ai choisi 5k / 20k / 60k pour observer un éventuel palier de performance quand on augmente la quantité.
quantities = {
    "5k": 5_000,
    "20k": 20_000,
    "60k": 60_000
}

results = []

# Baseline sans augmentation
# -> J'entraîne d'abord sans augmentation (10 époques, batch 128) pour avoir un point de comparaison.
print("== Baseline (aucune aug), 10 époques ==")
base_model = build_mnist_cnn()
base_model.fit(x_train_small, y_train_small, batch_size=128, epochs=10, verbose=1)
base_score = base_model.evaluate(x_test, y_test, verbose=0)
results.append(("none", "500", 10, base_score[1]))

# Entraînement avec augmentation
# Je génère N images synthétiques par époque via steps_per_epoch = N / batch_size.
EPOCHS = 10
BATCH = 128

for name, gen in augs.items():
    if name == "none": # -> Déjà fait ci-dessus.
        continue

```



```

gen.fit(x_train_small) # -> J'ajuste le générateur aux 500 images réelles.
for qty_name, qty in quantities.items():
    steps = int(qty / BATCH)
    model = build_mnist_cnn()
    flow = gen.flow(x_train_small, y_train_small, batch_size=BATCH, shuffle=True)
    print(f"\n== Aug: {name} | qty/epoch: {qty_name} | epochs: {EPOCHS} ==")
    # -> J'entraîne 10 époques à quantité fixée, pour comparer proprement les politiques/quantités.
    model.fit(flow, steps_per_epoch=steps, epochs=EPOCHS, verbose=1)
    score = model.evaluate(x_test, y_test, verbose=0)
    results.append((name, qty_name, EPOCHS, score[1]))

# Synthèse des résultats
# J'agrége tout dans un DataFrame, puis je trie par précision décroissante pour identifier la meilleure config.
import pandas as pd
df = pd.DataFrame(results, columns=["augmentation", "qty_per_epoch", "epochs", "test_acc"])
print("\nRésumé (trié par précision décroissante):")
print(df.sort_values("test_acc", ascending=False).to_string(index=False))

```

```

156/156 ————— 0s 1ms/step - accuracy: 0.7524 - loss: 0.7799

== Aug: medium | qty/epoch: 60k | epochs: 10 ==
Epoch 1/10
468/468 ————— 7s 5ms/step - accuracy: 0.1596 - loss: 2.2725
Epoch 2/10
468/468 ————— 0s 363us/step - accuracy: 0.2280 - loss: 2.1661
Epoch 3/10
468/468 ————— 0s 315us/step - accuracy: 0.4495 - loss: 1.9331
Epoch 4/10
468/468 ————— 0s 413us/step - accuracy: 0.5139 - loss: 1.5911
Epoch 5/10
468/468 ————— 0s 336us/step - accuracy: 0.5702 - loss: 1.2577
Epoch 6/10
468/468 ————— 0s 350us/step - accuracy: 0.6358 - loss: 1.0527
Epoch 7/10
468/468 ————— 0s 341us/step - accuracy: 0.7080 - loss: 0.9069
Epoch 8/10
468/468 ————— 0s 371us/step - accuracy: 0.7421 - loss: 0.8109
Epoch 9/10
468/468 ————— 0s 340us/step - accuracy: 0.8057 - loss: 0.6435
Epoch 10/10
468/468 ————— 0s 346us/step - accuracy: 0.7960 - loss: 0.6169

== Aug: heavy | qty/epoch: 5k | epochs: 10 ==
Epoch 1/10
39/39 ————— 7s 59ms/step - accuracy: 0.1143 - loss: 2.2913
Epoch 2/10
39/39 ————— 0s 5ms/step - accuracy: 0.2562 - loss: 2.2252
Epoch 3/10
39/39 ————— 0s 4ms/step - accuracy: 0.2611 - loss: 2.1525
Epoch 4/10
39/39 ————— 0s 4ms/step - accuracy: 0.3232 - loss: 2.0125
Epoch 5/10
39/39 ————— 0s 4ms/step - accuracy: 0.3605 - loss: 1.8352
Epoch 6/10
39/39 ————— 0s 4ms/step - accuracy: 0.4332 - loss: 1.7031
Epoch 7/10
39/39 ————— 0s 5ms/step - accuracy: 0.4955 - loss: 1.5295
Epoch 8/10
39/39 ————— 0s 4ms/step - accuracy: 0.5204 - loss: 1.3981
Epoch 9/10
39/39 ————— 0s 4ms/step - accuracy: 0.5538 - loss: 1.3065
Epoch 10/10
39/39 ————— 0s 4ms/step - accuracy: 0.5803 - loss: 1.1810

== Aug: heavy | qty/epoch: 20k | epochs: 10 ==
Epoch 1/10
156/156 ————— 7s 20ms/step - accuracy: 0.1667 - loss: 2.2909
Epoch 2/10
156/156 ————— 0s 1ms/step - accuracy: 0.2223 - loss: 2.2306
Epoch 3/10
156/156 ————— 0s 987us/step - accuracy: 0.2792 - loss: 2.1267
Epoch 4/10
156/156 ————— 0s 1ms/step - accuracy: 0.3839 - loss: 1.9599
Epoch 5/10
156/156 ————— 0s 1ms/step - accuracy: 0.4122 - loss: 1.8030
Epoch 6/10
156/156 ————— 0s 1ms/step - accuracy: 0.5036 - loss: 1.5451

```

J'affiche sous forme de courbe pour mieux voir l'évolution et comparer facilement avec la baseline

```

import matplotlib.pyplot as plt

# Je garde uniquement les lignes avec augmentation
dfp = df[df["augmentation"] != "none"].copy()

# Je convertis '5k', '20k', '60k' en valeurs numériques pour l'axe des x

```

```

map_qty = {"5k": 5000, "20k": 20000, "60k": 60000}
dfp["qty_num"] = dfp["qty_per_epoch"].map(map_qty)

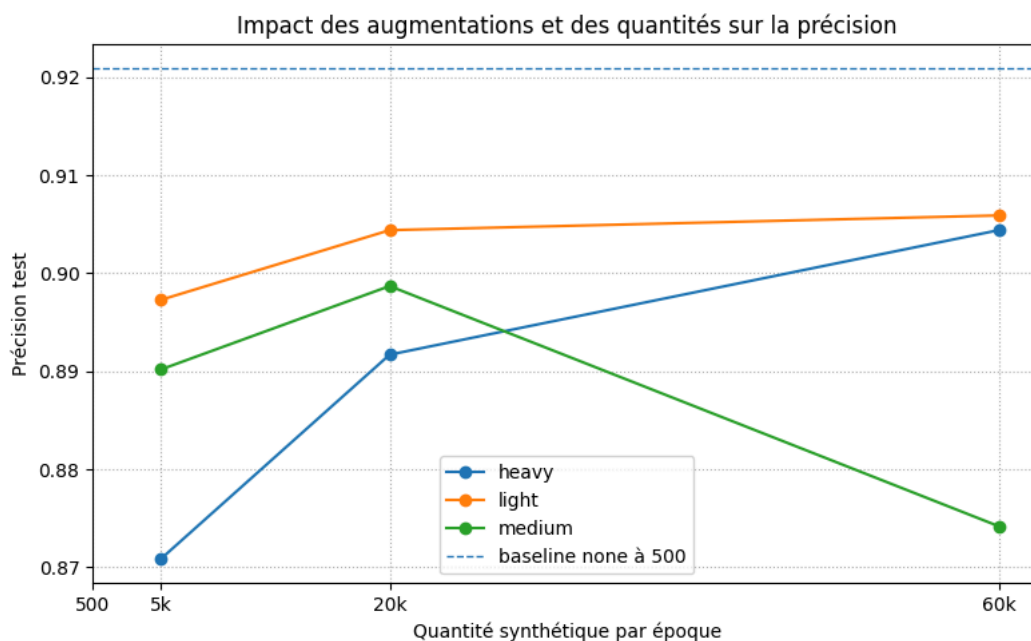
# Je trace une courbe par type d'augmentation (light, medium, heavy)
plt.figure(figsize=(8,5))
for aug, g in dfp.groupby("augmentation"):
    g = g.sort_values("qty_num")
    plt.plot(g["qty_num"], g["test_acc"], marker="o", label=aug)

# Je trace la baseline (aucune augmentation, 500 exemples réels)
base = float(df.loc[df["augmentation"]=="none", "test_acc"].iloc[0])
plt.axhline(base, linestyle="--", linewidth=1, label="baseline none à 500")

# Je configure les axes le titre et la légende
plt.xticks([500, 5000, 20000, 60000], ["500", "5k", "20k", "60k"])
plt.xlabel("Quantité synthétique par époque")
plt.ylabel("Précision test")
plt.title("Impact des augmentations et des quantités sur la précision")
plt.grid(True, linestyle=":")
plt.legend()
plt.tight_layout()

# J'affiche la figure
plt.show()

```



- Pour mesurer l'effet de l'augmentation sur MNIST, j'ai gardé le même modèle (10 époques, batch 128) avec 500 images réelles.
- J'ai testé trois niveaux (light, medium, heavy) et trois quantités (5k, 20k, 60k). Dans ce run, le meilleur score est sans augmentation : 0,9209.
- Les meilleures configs avec augmentation restent en dessous : light 60k = 0,9059, light 20k = 0,9044 et heavy 60k = 0,9044.
- Des réglages plus forts dégradent la précision (medium-60k = 0,8742, heavy-5k = 0,8709).

A faire : modifiez l'architecture pour la classification d'images naturelles et appliquez le au jeu de données Cifar10.

Pour des images naturelles, j'ai adapté mon réseau à CIFAR-10 (32x32, couleur) : convolutions 3x3, max-pooling, un peu de dropout et moyenne globale au lieu de Flatten. J'ai aussi mis une augmentation légère (miroir horizontal, petites rotations, zoom, contraste). En 20 époques (batch 128, Adam 1e-3), j'obtiens 0,8404, en validation (ép. 16), 0,8350 en fin d'entraînement, et 0,8228 en test. Ces changements aident le modèle à mieux capter les textures et couleurs tout en limitant le surapprentissage.

```

from tensorflow import keras
from tensorflow.keras import layers

# CIFAR-10 : classification d'images naturelles
# Objectif : j'adapte mon CNN (pensé pour MNIST) aux images couleur 32x32x3
# et j'applique une augmentation de données adaptée.

# 1) Données CIFAR-10
# Je charge le jeu et je normalise en [0,1].
(x_train, y_train), (x_test, y_test) = keras.datasets.cifar10.load_data()
num_classes = 10

```

```

x_train = x_train.astype("float32")/255.0
x_test  = x_test.astype("float32")/255.0

# 2) Augmentation adaptée aux images naturelles
# J'utilise des petites transformations réalistes : flip horizontal,
# légère rotation, zoom et contraste. Cela aide à la généralisation.
data_aug = keras.Sequential([
    layers.RandomFlip("horizontal"),
    layers.RandomRotation(0.05),
    layers.RandomZoom(0.1),
    layers.RandomContrast(0.1),
])

# 3) Modèle : mon CNN "MNIST", mais adapté à 32x32x3 et plus profond
# J'empile des blocs Conv(3x3) pour capter textures et couleurs.
# J'ajoute BatchNormalization pour stabiliser l'entraînement.
# J'utilise MaxPooling pour réduire la taille spatiale.
# J'ajoute du Dropout pour limiter l'overfitting.
# J'utilise GlobalAveragePooling2D à la place de Flatten + gros Dense
# pour réduire le nombre de paramètres et mieux généraliser.
model = keras.Sequential([
    layers.Input((32,32,3)),
    data_aug,

    layers.Conv2D(64, (3,3), padding='same', activation='relu'),
    layers.Conv2D(64, (3,3), padding='same', activation='relu'),
    layers.BatchNormalization(),
    layers.MaxPooling2D(),
    layers.Dropout(0.25),

    layers.Conv2D(128,(3,3), padding='same', activation='relu'),
    layers.Conv2D(128,(3,3), padding='same', activation='relu'),
    layers.BatchNormalization(),
    layers.MaxPooling2D(),
    layers.Dropout(0.3),

    layers.Conv2D(256,(3,3), padding='same', activation='relu'),
    layers.Conv2D(256,(3,3), padding='same', activation='relu'),
    layers.BatchNormalization(),
    layers.MaxPooling2D(),
    layers.Dropout(0.4),

    layers.GlobalAveragePooling2D(),
    layers.Dense(128, activation='relu'),
    layers.Dropout(0.5),
    layers.Dense(num_classes, activation='softmax')
])

# J'entraîne avec Adam (1e-3) et la loss adaptée aux labels entiers.
model.compile(optimizer=keras.optimizers.Adam(1e-3),
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])

# Je garde 20 époques (validation_split=0.1) pour suivre la val_accuracy.
model.fit(x_train, y_train, batch_size=128, epochs=20, validation_split=0.1, verbose=1)

# J'évalue sur le test set et j'affiche l'accuracy finale.
test_loss, test_acc = model.evaluate(x_test, y_test, verbose=0)
print("CIFAR-10 test acc:", round(float(test_acc),4))

```

```

Epoch 1/20
352/352 ————— 22s 48ms/step - accuracy: 0.3166 - loss: 1.8929 - val_accuracy: 0.1736 - val_loss: 3.7590
Epoch 2/20
352/352 ————— 16s 45ms/step - accuracy: 0.5315 - loss: 1.3086 - val_accuracy: 0.5694 - val_loss: 1.2872
Epoch 3/20
352/352 ————— 16s 46ms/step - accuracy: 0.6039 - loss: 1.1258 - val_accuracy: 0.6156 - val_loss: 1.1306
Epoch 4/20
352/352 ————— 16s 45ms/step - accuracy: 0.6600 - loss: 0.9927 - val_accuracy: 0.6796 - val_loss: 0.9593

```

Impossible d'établir une connexion avec le service reCAPTCHA. Veuillez vérifier votre connexion Internet, puis actualiser la page pour afficher une image reCAPTCHA.